



Actividad práctica en parejas

Objetivo: tomar un juego de oleadas en consola que tiene tres problemas de creación, el gestor de audio se abre tres veces, el héroe tiene un constructor de ocho parámetros y el nivel decide qué enemigo crear con un *switch* sobre un texto, y resolverlos con Singleton, Builder y Factory Method. Después, agregar un nivel nuevo sin tocar el proceso de la oleada.

Los archivos

El código de la actividad está en el repositorio de la cátedra. Si ya está clonado, alcanza con actualizarlo desde la carpeta del repositorio:

```
git pull
```

Si no, se clona una única vez:

```
git clone https://github.com/P00-CR/poo-2026.git
```

También está en archivo comprimido en el classroom. La carpeta de hoy es *13-creacionales/actividad*.

Es un juego de oleadas en consola, todo en el paquete *modelo*. *Enemigo* es la clase base de *Orco*, *Troll* y *Arquero*, y no se modifican. *GestorAudio* es la placa de sonido de verdad, con *javax.sound.sampled*, que viene con Java. Reproduce los archivos WAV de la carpeta *assets* y cuenta cuántas veces se abrió el dispositivo. Si la máquina no tiene audio, el juego sigue en silencio y el resto de la actividad no cambia. *Heroe* tiene nombre, vida, ataque, defensa y cuatro piezas de equipamiento opcionales, todo en un constructor. *Nivel* genera oleadas de enemigos según un texto que recibe al crearse. *Programa* arma todo, simula un ataque e imprime cuántos dispositivos de audio se abrieron.

Parte 1: ver el problema

1. Ejecutar *Programa*. Se escuchan tres sonidos, la música, el cuerno de la oleada y la espada, y la última línea dice que se abrieron tres dispositivos de audio. Buscar los tres *new GestorAudio()* y anotar en qué clases están.
2. Leer la línea de *Programa* que crea al héroe. Tiene ocho argumentos, un *null*, una cadena vacía y otro *null*. Sin mirar el constructor, decidir cuál de los cuatro últimos es la montura. Después mirar el constructor y verificar.
3. Leer *generarOleada()* en *Nivel*. Descomentar el bloque de la parte 3 al final de *Programa* y ejecutar. El pantano imprime *false* y ningún enemigo, sin ningún error, porque el *switch* no conoce "arquero". Volver a comentar el bloque.

Parte 2: los tres patrones

Un TODO por vez, en orden. Cada TODO indica en qué archivos se trabaja. Entre el TODO 3 y las subclases de *Nivel* el proyecto no compila: es esperable.



- TODO 1 (*GestorAudio.java*, *Heroe.java*, *Nivel.java*, *Programa.java*): *GestorAudio* pasa a ser Singleton, con el constructor privado, un atributo estático con la única instancia y *getInstancia()*. *Heroe* y *Nivel* dejan de crear el suyo y lo reciben por constructor. *Programa* lo obtiene una sola vez con *getInstancia()* y se lo pasa a los dos. Al ejecutar, la última línea tiene que decir que se abrió un dispositivo.
- TODO 2 (*Heroe.java*, *Programa.java*): el constructor de ocho parámetros se reemplaza por un *Builder* anidado. Lo obligatorio, el nombre y el gestor de audio, va en el constructor del *Builder*. Vida, ataque y defensa tienen valores por defecto, 100, 10 y 5, y un método cada uno. El equipamiento es opcional, con *conArma()*, *conArmadura()*, *conMontura()* y *conMascota()*, y por defecto es una cadena vacía. El constructor de *Heroe* pasa a ser privado, recibe el *Builder* y valida todo junto. En *Programa*, el héroe se construye con *new Heroe.Builder("Aragorn", audio)*, seguido de los métodos que hagan falta y *construir()*.
- TODO 3 (*Nivel.java*, *Programa.java* y dos clases nuevas): *Nivel* pasa a ser abstracta y deja de recibir el tipo de enemigo. El *switch* se reemplaza por una llamada a *crearEnemigo()*, un método abstracto y protegido. *NivelBosque* crea orcos y *NivelFortaleza* crea trolls, cada una con un constructor que recibe el gestor de audio y lo pasa a *Nivel* con su nombre. En *Programa*, el bosque se crea con *new NivelBosque(audio)*.

Con los tres TODO hechos, la salida es la misma que al principio salvo la última línea, un dispositivo en vez de tres.

Parte 3: el pantano

- TODO 4: agregar *NivelPantano*, que crea arqueros. Descomentar el bloque de la parte 3 al final de *Programa*, reemplazando la línea del *new* por *new NivelPantano(audio)*. La oleada tiene que imprimir *true* y cuatro enemigos.

Antes de seguir, comprobar que *Nivel.java* no cambió desde el TODO 3, y que *generarOleada()* no tiene una sola línea que nombre a *Orco*, *Troll* o *Arquero*.

Para discutir en la pareja y anotar:

- *GestorAudio* pide una línea de audio por sonido al crearse, y *reproducir()* espera a que el sonido termine antes de devolver. Si el juego del proyecto necesita música de fondo mientras se juega, ¿alcanza con este método? ¿Qué habría que cambiar?
- Después del TODO 1 hay un solo dispositivo. ¿Qué lo garantiza, el constructor privado o que *Programa* llame a *getInstancia()* una sola vez? ¿Qué se perdería si *Heroe* llamara a *GestorAudio.getInstancia()* por su cuenta en vez de recibirlo?
- En el TODO 2, ¿dónde quedó la validación? Si alguien escribe *new Heroe.Builder("", audio).construir()*, ¿qué pasa y en qué momento?
- El guardado de partida va a traer el nivel como texto, "pantano". ¿Factory Method sirve para reconstruirlo a partir del texto? ¿Qué forma de Factory corresponde ahí?
- De los tres patrones de hoy, ¿cuál resuelve algo que la inyección por constructor ya resolvía? ¿Cuándo vale la pena usarlo de todos modos?



- En el proyecto integrador, ¿dónde hay un *switch* antes de un *new*? ¿Qué constructor tiene más de cuatro parámetros? ¿Qué objeto no puede existir dos veces?